

1 Introduction

This document will contain the implementation details for the design learning platform. It will be used to outline the implementation before it is developed.

2 Components

WIP

In this section, I will outline the relationship between the following components and the practical steps needed to establish the relationship:

1. Artha - Meaningful Programming Language
2. DAL - Design Abstraction Language
3. CSM - Computable Semantic Model
4. CSP - Computable Semantic Primitives
5. Engine - Semantic Reasoning Engine
6. CLP - Compressed Log Processor
7. SI - Synthesized Implementation

When discussing this framework, the core principles revolve around meaning. So I think I will start by establishing that meaning is constructed by narratives that use the primitive meaningful building blocks. The narratives themselves have meaning and structure and that structure is used to automatically build a semantic model written in the design abstraction language. Since the meaning was built from computable semantic primitives, which are themselves established using the DAL, the semantic model becomes executable and becomes a computable semantic model.

Once meaning has been established, now it can be used in another narrative. It is a compositional system where meaning is composed through narratives and used to compose more meaning. Since the meaning is built from computable semantic primitives and it has an unambiguous structure, it can be synthesized into an implementation. In the process, the meaning of the software system has been automatically developed in a programming language.

The engine reasons about the meaning using the structure established by the computable semantic model. Since the structure is unambiguous and meaningful, it is able to reason about it to enforce its own definition of correctness and to also identify missing or incorrect meaning. Through the synthesis and instrumentation, the necessary information needed to replay the execution is preserved using domain specific compression and the engine is able to diagnose any failures and enable the meaning to expand through root cause analysis. This process automatically manages the software system and enables a deterministic learning loop.

Since the meaning of the participant is specified as part of the narrative, its domain structure can also be included in that meaning. So the meaning is what CLP uses to perform the domain specific compression, the domain structure is part of its meaning. If you imagine a story, the description of the character (which is part of the meaning) can be as detailed and unambiguous as needed, in this case, that description can be extended to include the domain structure of the participant.

The meaning can also tell you more about the data (ex. how it changes) so that it can be exploited to optimally compress this. There is a difference between knowing a value is a float and knowing that it is a GPS coordinates latitude or ambient temperature. It also makes the data itself meaningful, for example, if its GPS coordinates, you know when there is movement and when there is no movement. Also, since the meaning is already known, the queries will be meaningful and the ingestion performance can be improved. There are lots of implications for this but I'll discuss them later in the domain specific compression section. Ultimately, I just see meaning as the elimination of ambiguity in the domain (which CLP benefits from) and this framework provides the means to fully eliminate ambiguity in the meaning.

Next, I will discuss how each component can be practically implemented.

2.1 Design Abstraction Language

The Design Abstraction Language (DAL) itself is just a structure that can be used to construct the meaning of a design. I've written a basic AST library that can be used to establish blocks and commands with arguments, this is sufficient for now. Its the structures that are built with this that define the meaning.

Note: Since the design will be machine generated by constructing it from the artha specification, I can just use JSON, there is no need to use a custom language. I will modify this section later but for now this note will be sufficient.

2.1.1 Structure

The structural specification enables the the construction of a block using braces. For example, the block can be used as follows to specify control flow logic for a behavior.

```
1 select {
2     # Specify control flow logic
3 }
```

2.1.2 Commands or Attributes

The command structure itself is specified as shown below. Each command is specified and in the bracket the arguments are provided. There can be keyword commands used by the design and then there are custom commands that reference more meaning.

```
1 # Built In
2 design("Design Name")
3
4 # Custom Command
5 command(
6     number=123,
7     string="String",
8     arrayArg=[12,33,12],
9     objArg={"a":1}
10 )
11
12 # Reusing structure to specify attributes
13 property(
14     name="type"
15     description="Data type of participant (list, object, str etc)"
16     type="string"
17     get="getType"
18 )
```

2.1.3 Control Flow

The control flow structure itself is specified as specified below. Currently, only if statements are supposed but not else if or else blocks. I need to extend the library to support this.

```
1
2 # Currently supported
3 if (boolean) {
4
5 }
```

This basic structure makes it possible to specify all the necessary attributes without making the AST implementation itself complicated. As I work through the reasoning and build the CSM, I will be using

these structures specify the necessary meaning. If I run into features I can't support, I will return here and add to it.

2.1.4 Abstract Syntax Tree

The DAL script is parsed into an Abstract Syntax Tree(AST) that is then used by the validator. An example of a simple AST generated by a DAL script is provided below. In a full design specification, multiple dal files can be used to specify the behavior attributes, participant attributes etc.

2.1.5 Sample DAL Script

```
1 design("test")
2
3 behavior getName {
4     select {
5         goToBehavior(addBook)
6     }
7 }
8
9 behavior addBook {
10    #...
11    select {
12        goToBehavior(getName)
13    }
14 }
15
```

2.1.6 Sample AST

```
1 {
2     "type": "design",
3     "body": [
4         {
5             "type": "behavior",
6             "behaviorName": "getName",
7             "body": [
8                 {
9                     "type": "select",
10                    "args": [],
11                    "body": [
12                        {
13                            "type": "cmd",
14                            "command": "goToBehavior",
15                            "args": [
16                                {
17                                    "arg": null,
18                                    "type": "name",
19                                    "value": "addBook"
20                                }
21                            ]
22                        }
23                    ]
24                }
25            ]
26        },
27        {
28            "type": "behavior",
```

```

29     "behaviorName": "addBook",
30     "body": [
31         {
32             "type": "select",
33             "args": [],
34             "body": [
35                 {
36                     "type": "cmd",
37                     "command": "goToBehavior",
38                     "args": [
39                         {
40                             "arg": null,
41                             "type": "name",
42                             "value": "getName"
43                         }
44                     ]
45                 }
46             ]
47         }
48     ],
49     "name": [
50         {
51             "arg": null,
52             "type": "string",
53             "value": "test"
54         }
55     ]
56 ]
57 }
58 }

```

2.1.7 Modifying AST by injecting nodes

TODO — Implement feature to modify AST by injecting nodes

This is necessary because the structure of the describe will be generated from the Artha script and through the reasoning, the engine will make automatic modifications to the structure by enforcing the designs own meaning. I am not talking about the actual types of changes yet, just the ability to add nodes.

2.2 Narrative

In this section, I will build intuition for the role that each component plays by speaking generally about narratives using semantics used by humans in english. I will use it to demonstrate how the narrative can be constructed in a structured form and how that meaning can be used to construct more meaning. Then I will demonstrate how the meaning of narratives can be reasoned about to identify if it is internally consistent or if it has incorrect or missing meaning. Then I will discuss how the components outlined in this document work toward the same goal for software systems.

2.2.1 Library Manager

Consider the following narrative and participants in the narrative. In this case, the world is defined with the librarian and shelf. The user interacts with the world through the librarian and narratives unfold.

Narratives

The librarian presents the user with the option to add a book or get a book from the library.

If the user selects the option to add a book, the librarian accepts a book by asking the user for the name and content. Then the librarian reads the first letter of the books name to place the book on the relevant shelf. Then the user is once again asked which action they want to take.

If the user selects the option to get a book, the librarian asks the user for the book name. The librarian reads the first letter of the book name and accesses the book from the correct slot on the shelf. After the librarian provides the user the book, the user is once again asked which action they want to take.

Librarian

The librarian can present the user with a menu.
The librarian executes the users choice.
The librarian adds a book to the shelf.
The librarian gets a book from the shelf.

Menu

The menu provides the user with the option to submit a book or get a book.

Book

A book has a name and it has contents. Both the name and contents contain letters.

Shelf

A shelf has slots for books. The slots are labeled using a letter.

In this example, I am using the meaning of words in the english language. As part of human communication, we use shared meaning to coordinate. When I say the word Jump, that has a specific meaning. It includes the sequence of actions involved in propelling ones self off the ground or how it looks when jumping happens.

When I say the word Lion, that includes the visual appearance of the lion, the sound, its strength etc. All of that is compressed into the meaning of a lion.

In my narrative there are words which have meaning and we refer to these as nouns (including adjective and adverbs which modify or describe the noun). Then there are functional words like conjunctions (and, if, then etc.) and prepositions (in, on etc.) that connect the meaning together. The functional words create a narrative using the meaning.

I think this exercise is helpful because it helps build the relationship between meaning, narratives and ambiguity. In human communication, when I say, "the librarian added a book to the shelf", I am communicating a narrative that has been constructed using the meaning of the librarian, book, shelf while connecting them with an action (added a book) and functional words. When there is ambiguity in the meaning of the librarian, book or shelf, it creates potential for failure to realize the narratives intentions. Is it possible to add this book to the shelf?

This can only be answered by reasoning about the meaning of the book, shelf and the action of adding a book to the shelf. For example, if the shelf has slots of a certain size and the book is larger than the size of the slot, then the intention will fail. By reasoning about the world, the narratives can be modified so that the intentions will succeed by preventing narratives that are semantically invalid. It can also be seen that if two narratives are interacting, the success of that interaction is dependent on shared meaning. Without that shared meaning it will be impossible to reason about whether the interaction will succeed.

To further discuss reasoning about the meaning of this world, I will use a diagram of the narrative by representing the narrative as behaviors of a closed semantic world, which can also be referred to as a design. Here a behavior is a semantic unit that transforms the world state in an unambiguous way. So the narrative can also be described as the behavior of the design.

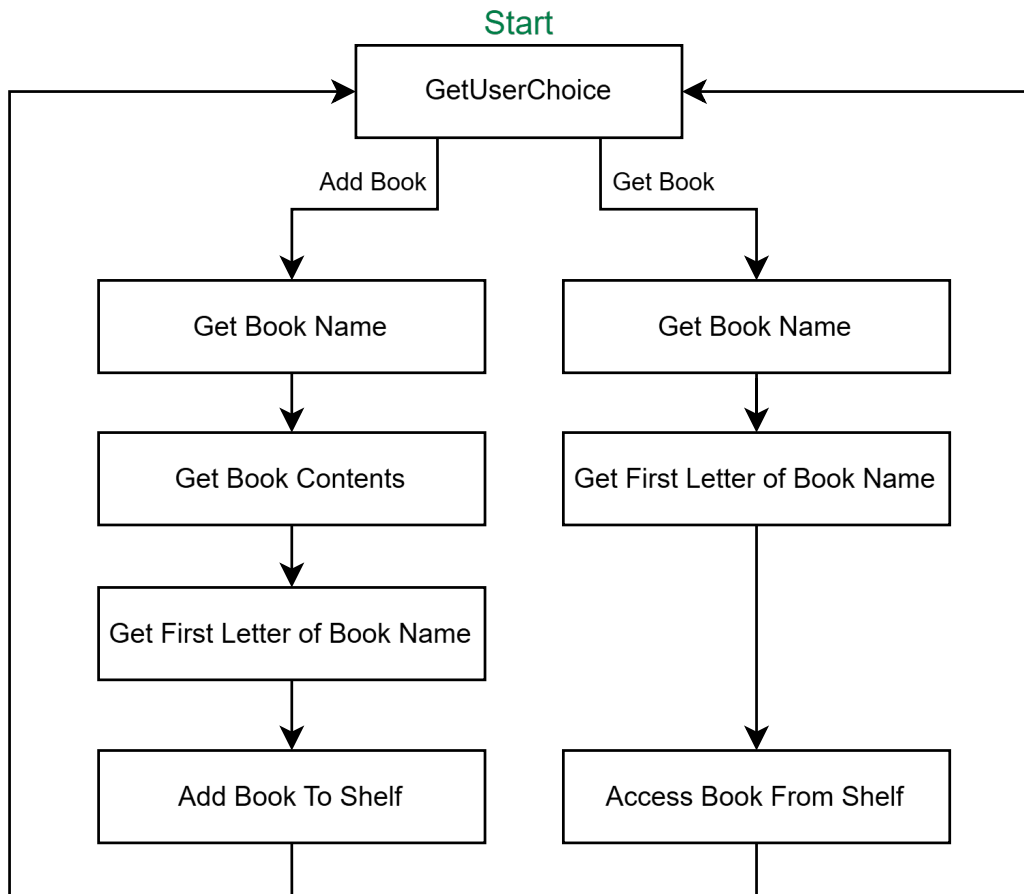


Figure 1: Library Manager Diagram

Using the structure of the design, we can reason about its meaning. First, it is evident from the design that the narratives establish the designs intentions. A design is said to be internally inconsistent if it makes assumptions that contradict its own meaning. As a result of this contradiction, it will be unable to realize its intentions because it permits conditions that make realizing its intentions not possible. For example, in this design, there is a behavior in which the first letter of the books name is accessed. However, the design accepts a book with no letters in its name with the intention of using the first letter of the book name.

When a book with no letters in its name is accepted, the design is said to be in a semantically invalid state because the designs intentions cannot be realized. In order for it to realize its intentions, it must modify its behavior only accept books with letters in its name. Through this process, it can be said that the predicted failure is resolved by preventing the semantic invalidity from persisting. This process of identifying the failure can be described as reasoning about the meaning of world.

Through the reasoning the design any internal inconsistency in the design can be eliminated and all known failures can be eliminated. However, when the design participates in reality, reality can refute the designs assumptions. This results in the design learning new meaning that it was previously unaware of and then through reasoning, the failure can be eliminated.

For example, in this example, its possible the shelf can only hold 100 books after which it will break. A failure will result in root cause analysis and the new semantics that is learnt will include the number of books in the shelf, the awareness of the maximum number of books and a refusal to add more books. Then the same process can be carried out and through reasoning any new failures can be eliminated.

In this section, I was being very general with the language and meaning I used just to build intuition for the relationship between reasoning, meaning, ambiguity and the the design. In the next section, I will explain move onto software systems and how they can be reasoned about completely by making their meaning entirely unambiguous by construction.

More to come.

2.2.2 Meaning of Software System

Unlike humans communication which has many factors that influence it, in the world of software systems, meaning can be made entirely unambiguous by construction. The design of a software system is a closed semantic world and the meaning of that world can be made entirely unambiguous and computable by constructing the meaning from computable semantic primitives that have unambiguous invariants. This allows an engine to completely reason about the world and diagnose failures in the construction of the world that are a result of missing or incorrect meaning.

In the next section, I will discuss how the computable semantic primitives are defined and how they establish the atomic meaning of the world. After that, I will discuss how Artha uses the semantic toolkit to establish narratives and how the world is automatically constructed using the design abstraction language. Finally, I will discuss how the engine reasons about the meaning of the world using the structure.

2.3 Computable Semantic Primitives

I think this is the best place to start because it introduces how meaning is established with the DAL while also introducing the first entries into the semantic toolkit that can be used to construct narratives. So next, I will present the CSP for add and then I will discuss each part of it.

Meaning of Add

```
1  design("add")
2  type("primitive")
3
4  description("Adds two numbers and produces a result. The numbers must have type float
   ↪ or int and the result will be float or int type.")
5
6  narrative_structure {
7    # Example: store result of add cost_of_bagel and cost_of_coffee in total_cost
8    narrative_syntax(f"store result of add {addend1} and {addend2} in {resultName}")
9    success_narrative(f"Added {addend1} to {addend2} and resulted in {result}.")
10   fail_inv_id_1(f"Failed to add the two numbers because addend1 was not a float or
   ↪ int.")
11   fail_inv_id_2(f"Failed to add the two numbers because addend2 was not a float or
   ↪ int.")
12 }
13
14 participants {
15   participant(name="addend1", required="true")
16   participant(name="addend2", required="true")
17   participant(name="resultName", required="true")
18 }
19
20 returns {
21   participant(
22     name="result",
23     type="float" or "int",
24     description="Result of the addition"
25   )
26 }
27
28 behavior add {
29   # Get the addends from the world state
30   getFromWorldState(name=addend1, role="addend1")
31   getFromWorldState(name=addend2, role="addend2")
32   getFromWorldState(name=resultName, role="resultName")
33
34   # adds the two values and store result in the value of resultName
35   _add(result, addend1, addend2)
36
37   # Update the world state
38   updateWorldState(
39     name=resultName,
40     value=result
41   )
42 }
43
44 run (add)
45
46
47 invariants {
```

```

48     assert({
49         id:1,
50         participants: ["addend1"],
51         property: "type"
52         condition: "float" || "int"
53         description: "Addend1 must be a whole number or float.",
54         invalidValues: [
55             {
56                 value: "string_value"
57                 description: "A string is provided as an addend."
58             }
59         ]
60     }),
61     assert({
62         id:2,
63         participants: ["addend2"],
64         property: "type"
65         condition: "float" || "int"
66         description: "Addend2 must be a whole number or float.",
67         invalidValues: [
68             {
69                 value: "string_value"
70                 description: "A string is provided as an addend."
71             }
72         ]
73     })
74 }
75 }

```

Realization of meaning of transformation Add

```

1 def add(addend1, addend2):
2     return addend1 + addend2

```

Realization of attribute type

```

1 def getType(value):
2     return type(value).__name__

```

2.3.1 Transformation and Realization

WIP

Given the necessary participants, the CSP itself is a meaningful transformation that results in an unambiguous world state after the transformation. The meaning of that transformation is established by the CSP but the realization of that transformation is established in the implementation that is invoked.

This is an important point to highlight because it is this property that enables the meaning of the world to be synthesized directly into an implementation. While before the programming language would be used to implement the intended meaning of the system, now the meaning is used to automatically synthesize the implementation because the meaning is made computable through the CSP's.

Now, "add" is now part of the semantic toolkit and it can be used to construct more meaning by using it in narratives. When the CSM is constructed from the artha script, the add transformation will be a transformation in the world and it will get synthesized using the provided implementation.

Since the meaning of add includes the invariants which establish its correctness, the engine can reason about

the narratives to establish the conditions under which this transformation will succeed. This is a direct result of establishing the meaning of the transformation and reasoning about it using its context in the narratives of the design.

As part of the invariant definition, specific attributes of the participant are bounded to certain values. For example, the participants in the add transformation are expected to have a type attribute and the realization of that attribute is invoked through the implementation. In this sense, the meaning of that attribute can be used to reason about whether the transformation will succeed.

2.3.2 Narrative Syntax

The narrative syntax is how the transformation can be used in an Artha script. The script itself is tokenized and using the syntax established for each primitive it can be parsed. The parsed syntax is also used to enforce the invariants because it establishes the meaning of the participants in this transformation.

2.4 Artha

In this section, I will introduce the meaningful programming language called Artha. I already established how the computable semantic primitives are built using the DAL and how the meaning established by those primitives is part of the semantic toolkit. Now, I will discuss how I can build narratives from those primitives to create more meaning.

In an earlier section, using the english language, I demonstrated how narratives can be constructed by combining meaning and functional words. Here, I transition to the world of software systems and establish the language needed to construct narratives from computable meaning. I will demonstrate how that narrative's structure can be constructed using the DAL and in the next section, I will describe how an engine can be used to reason about it.

2.4.1 Primitive Syntax and Constructed

The CSP's define the syntax that can be used in the Artha script. These can be used in conjunction with the functional syntax to create constructed syntax. Then the primitive, functional and constructed syntax can be used together to more meaning and establish more constructed syntax.

add the bill

Initialize bill_sum to 0. bill_sum is of type float.

For each participant in group add participant_bill and bill_sum.

In this example, the constructed syntax **add the bill** is constructed by establishing the narrative that it represents. Now add the bill can be used in another narrative.

If diners request the bill then add the bill then hand bill to diners.

2.4.2 Functional Syntax

In this section, I will begin introducing the functional syntax that must be used to construct the narratives. Using the example below, I will introduce the if, then and while functional units.

For basket has books get a book from basket **then if** book name equals "harry potter" add book to the shelf

The sudo code of how it will be parsed is presented below:

```
1 while (basket has books) {  
2     get a book from the basket  
3     if (book name equals "harry potter") {  
4         add book to the shelf  
5     }  
6 }
```

The DAL representation would be as show below, engine will reason about this:

```
1 behavior basketHasBooks {  
2     hasBooks = basketHasBooks(basket)  
3     if (hasBooks){  
4         goToBehavior("invoke_getBookFromBasket")  
5     }  
}
```

```

6      }
7
8      behavior invoke_getBookFromBasket {
9          book = getBookFromBasket()
10         if (book.name === "harry potter") {
11             goToBehavior("invoke_addBookToShelf")
12         }
13     }
14
15     behavior invoke_addBookToShelf {
16         addBookToShelf(shelf, book)
17         goToBehavior("basketHasBooks")
18     }

```

Synthesized Implementation (in python):

```

1     while (basketHasBooks()):
2         book = getBookFromBasket()
3         if (book["name"] == "harry potter"):
4             addBookToShelf(book)
5
6

```

2.4.3 If

2.4.4 Then

2.4.5 While